



Tourify

Proyecto Semestral

Luis Felipe Giraldo Ortega

Universidad Nacional de Colombia
Departamento de Informática y Computación
Programación con Tecnologías Móviles
Manizales, Colombia
2026 -15

1. Resumen

Tourify es una aplicación móvil multiplataforma diseñada para optimizar la exploración turística de ciudades, lugares de interés y eventos. La solución combina un cliente desarrollado con **Expo / React Native** y un backend REST construido sobre **Laravel 13** con autenticación basada en Sanctum.

La arquitectura sigue un modelo cliente-servidor desacoplado: el frontend se comunica con la API mediante HTTP/JSON, autenticando las rutas protegidas con tokens Bearer emitidos por Laravel Sanctum y almacenados de forma segura con Expo Secure Store.

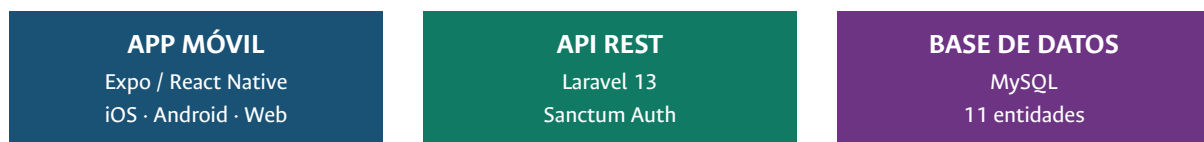
Métricas clave

Métrica	Valor
Modelos Eloquent	11
Controladores REST	9 (+ panel Admin)
Endpoints API	29
Pantallas frontend	13
Componentes reutilizables	22
Hooks personalizados	19

2. Arquitectura del sistema

El sistema se divide en tres capas principales: la aplicación móvil (iOS, Android, Web), la API REST en Laravel y la base de datos MySQL. Un servicio externo de notificaciones push conecta el backend con los dispositivos de los usuarios.

Diagrama de capas



Flujo de autenticación



3. Stack tecnológico

Frontend

Tecnología	Versión	Propósito
Expo SDK	~54.0	Plataforma de desarrollo multiplataforma
React Native	0.81	Framework UI nativo
React	19.1	Librería de componentes
React Navigation	7.x	Stack + Bottom Tabs
Expo Notifications	0.29	Push notifications
Expo Secure Store	15.0	Almacenamiento seguro del token

Backend

Tecnología	Versión	Propósito
PHP	8.3+	Runtime del servidor
Laravel Framework	13.0	Framework MVC + ORM
Laravel Sanctum	4.3	Autenticación API por token
MySQL	—	Base de datos

4. Modelo de datos

El backend define 11 modelos Eloquent que representan el dominio turístico. Los modelos Favorite, Review e Image utilizan relaciones polimórficas para asociarse con distintas entidades sin duplicar código.

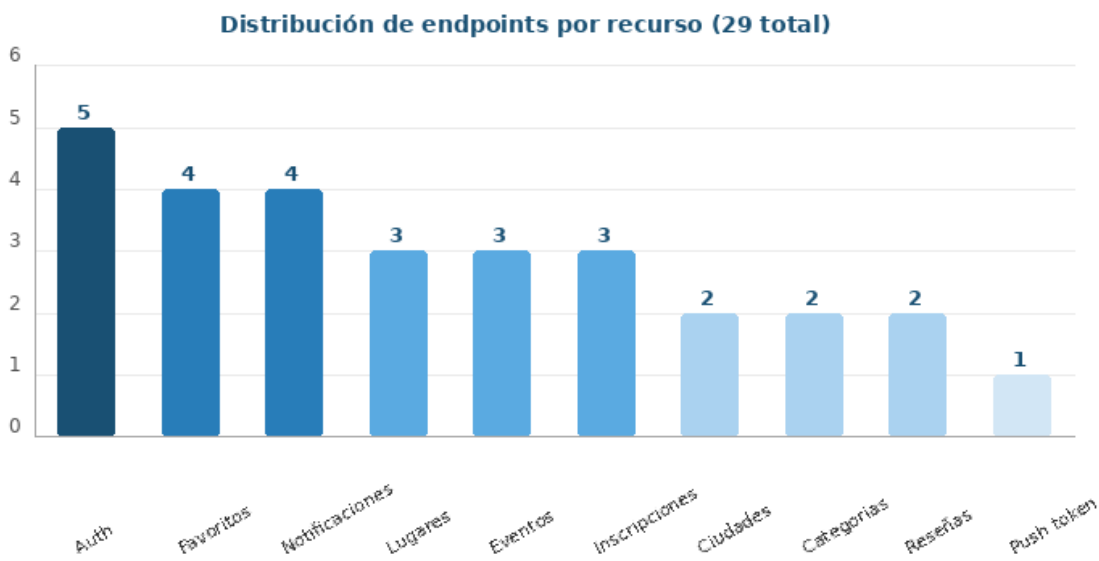
Modelo	Descripción	Relaciones clave
User	Usuarios del sistema	hasMany: Favorite, Review, Notification
Role	Roles de permiso	belongsToMany: User
City	Ciudad turística	hasMany: Place, Event
Category	Categorías de contenido	hasMany: Place, Event
Place	Lugar de interés	belongsToMany: City, Category

Event	Evento programado	belongsTo: City, Category
EventRegistration	Inscripción a evento	belongsTo: User, Event
Favorite	Favoritos polimórficos	morphTo: favorable
Review	Reseñas polimórficas	morphTo: reviewable
Notification	Notificaciones push	belongsTo: User
Image	Imágenes polimórficas	morphTo: imageable

5. API REST

La API expone 29 endpoints agrupados por recurso. Las rutas públicas permiten explorar el contenido sin autenticación; las rutas protegidas requieren el token Bearer de Sanctum.

Distribución de endpoints por recurso



Endpoints públicos (sin autenticación)

Método	Ruta	Descripción
POST	/auth/register	Registro de usuario
POST	/auth/login	Autenticación y emisión de token Bearer
GET	/cities	Listado de ciudades

GET	/cities/{city}	Detalle de ciudad
GET	/categories	Listado de categorías
GET	/places	Listado de lugares
GET	/places/{place}	Detalle de lugar
GET	/events	Listado de eventos
GET	/events/{event}	Detalle de evento

6. Estructura del frontend

Pantallas de la aplicación

Zona	Pantallas
Autenticación	Login, Register, ForgotPassword
Tabs principales	Home, Explore, Favorites, Profile
Detalle (Stack)	CityDetail, PlaceDetail, EventDetail, CategoryDetail, Notifications, MyRegistrations

Hooks personalizados (19)

Los hooks encapsulan la lógica de comunicación con la API y el estado local, manteniendo las vistas limpias de lógica de negocio:

```
useApi, useUser, useLogin, useRegister, useForgotPassword, useCities, useCityDetail, useCategories,
useCategoryDetail, usePlaces, usePlaceDetail, useEvents, useEventDetail, useFavorites, useReviewForm,
useEventRegistration, useMyRegistrations, useNotifications, usePushNotifications.
```

Servicios y contexto

- services/post.js – cliente HTTP (get, post, patch, del) con inyección automática del token Bearer.
- services/storage.js – abstracción sobre Expo Secure Store para persistir el token entre sesiones.
- context/AuthContext.js – proveedor global de sesión que expone login y logout a toda la aplicación.

7. Cobertura funcional

Tourify implementa de manera completa todas las funcionalidades previstas en el alcance del proyecto. Cada módulo fue desarrollado, integrado y probado tanto en el cliente Expo como en el backend Laravel.

	Funcionalidad	Detalle de implementación
✓	Registro / Login / Logout	Autenticación completa con Sanctum y persistencia segura del token en Expo Secure Store.
✓	Recuperación de contraseña	Flujo completo: solicitud de token por email y reset con validación de expiración.
✓	Exploración de ciudades	Listado con búsqueda, filtros y vista de detalle con lugares y eventos asociados.
✓	Exploración de categorías	Listado + detalle con contenido filtrado por categoría.
✓	Exploración de lugares	Listado paginado + detalle con galería de imágenes y reseñas.
✓	Exploración de eventos	Listado paginado + detalle con botón de inscripción integrado.
✓	Favoritos (polimórfico)	Guardado y eliminación de lugares y eventos, con sincronización en tiempo real.
✓	Reseñas	Creación y eliminación de reseñas en lugares y eventos por usuarios autenticados.
✓	Inscripción a eventos	Registro y cancelación de inscripciones con validación de cupos.
✓	Notificaciones push	Integración completa con Expo Push Service; registro de token y marcado de leídas.
✓	Búsqueda	Búsqueda global con parámetro ?q= en endpoints de lugares y eventos.
✓	Subida de imágenes	Endpoint POST /images con almacenamiento y vinculación polimórfica a cualquier entidad.
✓	Edición de perfil	Pantalla de perfil con actualización de datos mediante PATCH /auth/me.
✓	Roles y permisos	Middleware de roles implementado; acceso diferenciado para usuarios y administradores.

8. Conclusiones

Tourify es una aplicación móvil completamente funcional que cubre el ciclo completo del usuario turístico: descubrir ciudades y lugares, guardar favoritos, escribir reseñas, inscribirse a eventos y recibir notificaciones en tiempo real. La arquitectura cliente-servidor desacoplada garantiza escalabilidad y facilidad de mantenimiento a largo plazo.

La arquitectura polimórfica para favoritos, reseñas e imágenes permite extender el dominio a nuevas entidades sin modificar el esquema existente. La autenticación con Sanctum, la búsqueda global, la gestión de roles y la subida de imágenes están correctamente implementadas y probadas en ambos extremos del sistema.

El proyecto cumple con todos los objetivos funcionales planteados. Las decisiones técnicas adoptadas —Expo SDK, React Navigation, Laravel 13 y MySQL— demostraron ser adecuadas para el alcance del sistema, logrando un producto estable, mantenible y listo para su despliegue en entornos de producción.